Tutorial 2 (part 2)

qq. 2.2 (4), 2.3

SC1006, CZ1106

2.2 Arithmetic and Logical Instructions

(4) With the aid of the Shift operation, give the ARM instruction(s) that will multiply the **signed** content in register **R0** by the constant value **9**.

The value 9A is the same as (8A + A). By using this approach, we can use the logical shift left operation to do a 2^N multiplication of 8, where N is 3 (i.e. 3 bit shift left). The instruction that performs this is given by:

```
ADD R0,R0,R0,LSL #3    ;R0 = 8xR0 + R0 = 9R0
```

$3=3*2^0=$ 3: 0 0 0 1 1
$3*2=3*2^1=$ 6: 0 0 1 1 0 (left shift 1 position)
$3*4=3*4^2=$ 12: 0 1 1 0 0 (left shift by 2 positions)
$3*9=3*8+3=3*2^3+3=27$: 1 1 0 0 0 + 0 0 0 1 1 = 1 1 0 1 1 (left shift by 3 positions followed by addition)

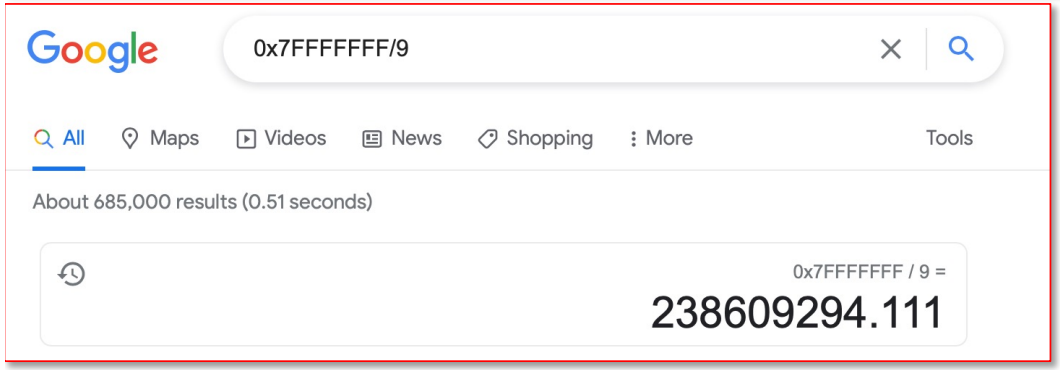
2.2 Arithmetic and Logical Instructions

(4) With the aid of the Shift operation, give the ARM instruction(s) that will multiply the **signed** content in register **R0** by the constant value **9**.

(i) Give the largest positive signed 32-bit value in **R0** that will not result in an overflow after this operation.

The largest positive 32-bit value is **0x7FFFFFFF**. We can find the largest value that can be multiplied by 9 without overflow by dividing this largest value by 9 and discarding the remainder (i.e. round down).

0x7FFFFFFF/9 = 0xE38E38E (238,609,294)



2.2 Arithmetic and Logical Instructions

(4) With the aid of the Shift operation, give the ARM instruction(s) that will multiply the **signed** content in register **R0** by the constant value **9**.

(i) Give the largest positive signed 32-bit value in **R0** that will not result in an overflow after this operation.

(ii) Suggest how this idea could be extended to multiply the 32-bit content in register **R0** with the constant value **7**.

Using the same idea, we compute 7A using (8A – A). However, we need to decide if we do subtraction using **RSB** or **SUB**. Below are the ways these subtraction operation work:

SUB dest,op1,op2 gives **dest = op1 – op2**

RSB dest,op1,op2 gives **dest = op2 – op1**

Since the shift operation can only be applied to operand op2, we need to use **RSB** to get the value of 7A.

```
RSB R0,R0,R0,LSL #3   ;R0 = 8xR0 - R0 = 7R0
```

2.3 ARM Assembly Program Analysis – Comparison and Counting

Fig. 3 shows an ARM assembly language program that scans through a sample of average daily temperature readings stored consecutively as unsigned 8-bit numbers in data memory starting at address **`0x102`** onwards. Two byte-sized memory variables **`HotDays`** and **`DaySum`** are located at addresses **`0x100`** and **`0x101`** respectively. The end of valid temperature readings in the array is detected when the decimal value of -128 is encountered.

Note: Replicate the program in Fig. 3 using the partial completed VisUAL ARM assembly program template ***Tutorial 2_3-Template***. Analyse the operation of the program by executing the code. All memory values shown in Fig. 3 will be pre-loaded into their respective memory locations by the given DCD directives.

1. *Understand how looping structure can be constructed using conditional branch instructions.*
2. *Understand how consecutive elements in an array can be accessed using register indirect.*
3. *Understand how compare can be used to detect similar values or values that are higher than a specific threshold value.*
4. *Understand how to convert between decimal and hexadecimal values.*
5. *Learn to optimise codes by avoiding unnecessary memory access and using appropriate addressing modes available in the ARM instruction set architecture.*

(1) Based on the hexadecimal memory contents shown in Fig. 3, compute the hexadecimal contents in registers **R0**, **R1**, **R2**, **R3**, **R4** and byte-sized memory variables **`HotDays`** and **`DaySum`** at the end of program execution. Assume execution begins at instruction label **`Start`** and ends at the program termination directive **`END`**.

(2) Describe the operation of the assembly language program in Fig. 3 by filling in appropriate comments for each line of instruction? In brief, what does this program do?

The program computes the total number of days (**`DaySum`**) by counting number of byte-sized values retrieved before the value -128 (**`0x80`**) is encountered. It also counts the number of values that are larger than or equal to the decimal value 36 (i.e. **`0x24`**) and leave the result in memory variable **`HotDays`**. The relevant comments have been added to **Figure 3_Ans**.

```
Mem_100     DCD   0x32240000 ; pre-load values into data memory
Mem_104     DCD   0x22282317
Mem_108     DCD   0x17208013
Mem_10C     DCD   0x2D142580
Start       MOV   R1,#0x100  ; load mem data start address into R1
            MOV   R4,#0x100  ; load address of HotDays into R4
            MOV   R0,#0      ; clear register R0 to zero
        ➡   STRB  R0,[R4]    ; clear HotDays mem variable to 0
            ADD   R1,R1,#2   ; R1 to point to start of array
Loop    ➡   LDRB  R3,[R1]    ; get current array element into R3
        ➡   CMP   R3,#0x80   ; check if it is the terminator 0x80
        ➡   BEQ   Done       ; if yes, then jump to Done
            ADD   R0,R0,#1   ; increment day counter R0
        ➡   LDRB  R3,[R1]    ; get current array element into R3
        ➡   CMP   R3,#36     ; compare it with the threshold 0x24
        ➡   BHS   HotFound   ; branch to HotFound if >= 0x24
            ADD   R1,R1,#1   ; inc to point to next array element
            B     Loop       ; branch back to start of Loop
HotFound➡   LDRB  R2,[R4]    ; get mem variable HotDays into R2
            ADD   R2,R2,#1   ; increment value in R2
        ➡   STRB  R2,[R4]    ; write back to mem variable HotDays
            ADD   R1,R1,#1   ; inc to point to next array element
        ➡   B     Loop       ; branch back to start of Loop
Done        ADD   R4,R4,#1   ; increment pointer R4 to point to mem variable DaySum
        ➡   STRB  R0,[R4]    ; copy day counter register R0 into mem variable DaySum
            END
```

Address	Contents
0x100	0xFF
0x101	0xFF
0x102	0x24
0x103	0x32
0x104	0x17
0x105	0x23
0x106	0x28
0x107	0x22
0x108	0x13
0x109	0x80
0x10A	0x20
0x10B	0x17
0x10C	0x80
0x10D	0x25
0x10E	0x14
0x10F	0x2D

R0 = 0x00000007 (The total number of days checked)
R1 = 0x00000109 (Address of the last array element accessed before termination of loop)
R2 = 0x00000003 (The total number of days exceeding temperature of 36)
R3 = 0x00000080 (Last value read from the array before termination of loop)*
R4 = 0x00000101 (Address of DaySum memory variable)
HotDays (0x100) = 0x03 (Same as register R2)
DaySum (0x101) = 0x07 (Same as register R0)

*Note: LDRB does zero-extension of the byte-sized value to a 32-bit value in the register.

STRB Store Register Byte (register) calculates an address from a base register value and an offset register value, and stores a byte from a register to memory.
LDRB – loads a byte from memory and writes it to the register
CMP - compare
BEQ – branch is equal
BHS – branch if higher or the same (unsigned)
B - branch

(3) Modify the program to speed up its execution. You can evaluate how well you have optimized your code by the reduction in clock cycle count whilst still getting the same results in all the registers and memory variables **HotDays** and **DaySum**. The given program takes 127 clock cycles to execute but it is possible to reduce it to 93 cycles!

```
Start       MOV     R4,#0x100      ; initialise R4 to point to start of memory data 0x100
            ADD     R1,R4,#1       ; initialise array pointer R1 to point to 0x101
            MOV     R0,#0          ; clear DaySum counter register R0
            MOV     R2,R0          ; clear HotDays counter register R2
Loop        LDRB    R3,[R1,#1]!    ; use index with pre-increment to access next element
            CMP     R3,#0x80       ; compare array element in R3 with terminator 0x80
            BEQ     Done           ; branch to Done if equal to terminator
            ADD     R0,R0,#1       ; increment DaySum counter register R0
            CMP     R3,#36         ; compare array element in R3 with threshold 0x24
            BHS     HotFound       ; branch to HotFound if >= 0x24
            B       Loop           ; branch back to Lopp
HotFound    ADD     R2,R2,#1       ; increment HotDays counter register R2
            B       Loop           ; branch back to Lopp
Done        STRB    R2,[R4]        ; store R2 into HotDays memory variable at 0x100
            STRB    R0,[R4,#1]!    ; store R0 into DaySum mem var at 0x101 and inc R4
            END
```

(4) It was observed that if temperature samples had negative values (e.g. -1 or **0xFF**), the results obtained were incorrect. Could you suggest a reason for this error and how it can be rectified in the program shown in Figure 3?

Suggested solution:

The problem with the **LDRB** instruction, is that it will zero-extend the byte read from memory into a 32-bit value in the register **R3**. This means a negative byte-sized value like **0xFF** (-1) will turn into the 32-bit word **0x000000FF**, which is a positive value.

In order to address this issue, the byte-sized value fetched from memory must be sign-extended into 32-bits instead of zero-extended. Unfortunately, the **LDRSB** mnemonic is not supported in the VisUAL ARM simulator. This operator will sign-extend the byte to 32-bits in the destination register.

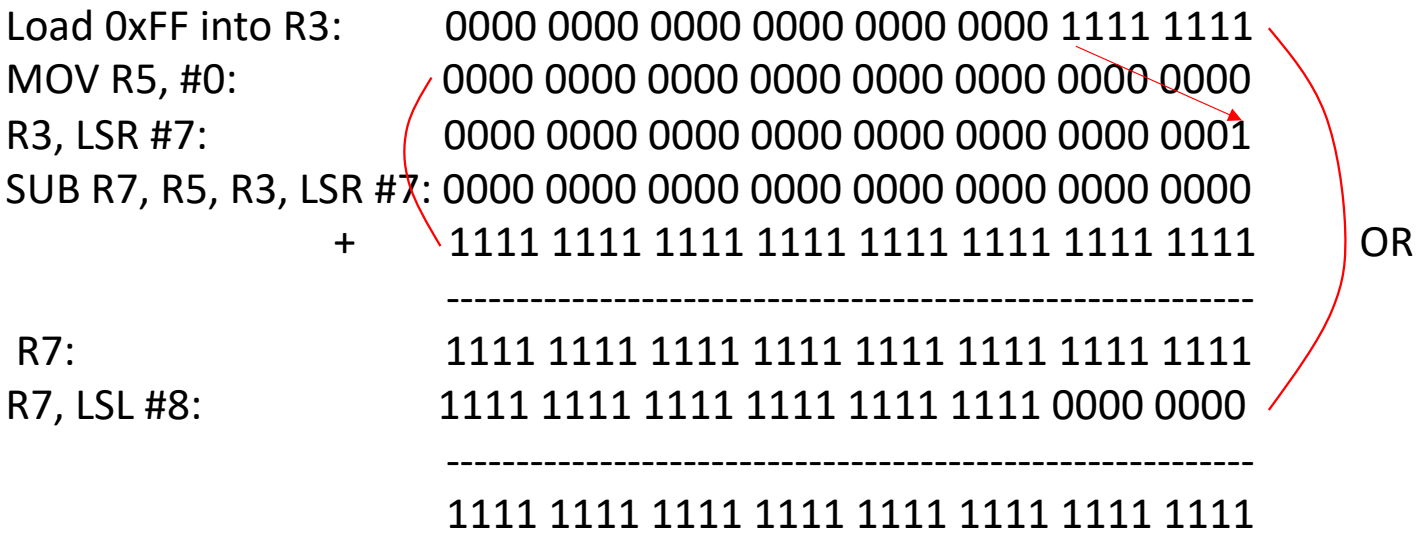
The following code segment that is to be inserted just before **CMP R3,#36** should be able to sign-extend the zero-extended byte read from memory into **R3**:

```
MOV   R5,#0                ; create zero value register
SUB   R7,R5,R3,LSR #7      ; get all 1's(-ve) or 0's(+ve) by doing  0 - (R3_byte >> 7)
ORR   R3,R3,R7,LSL #8      ; OR with byte higher order 1's or 0's from bit 8 onwards
```

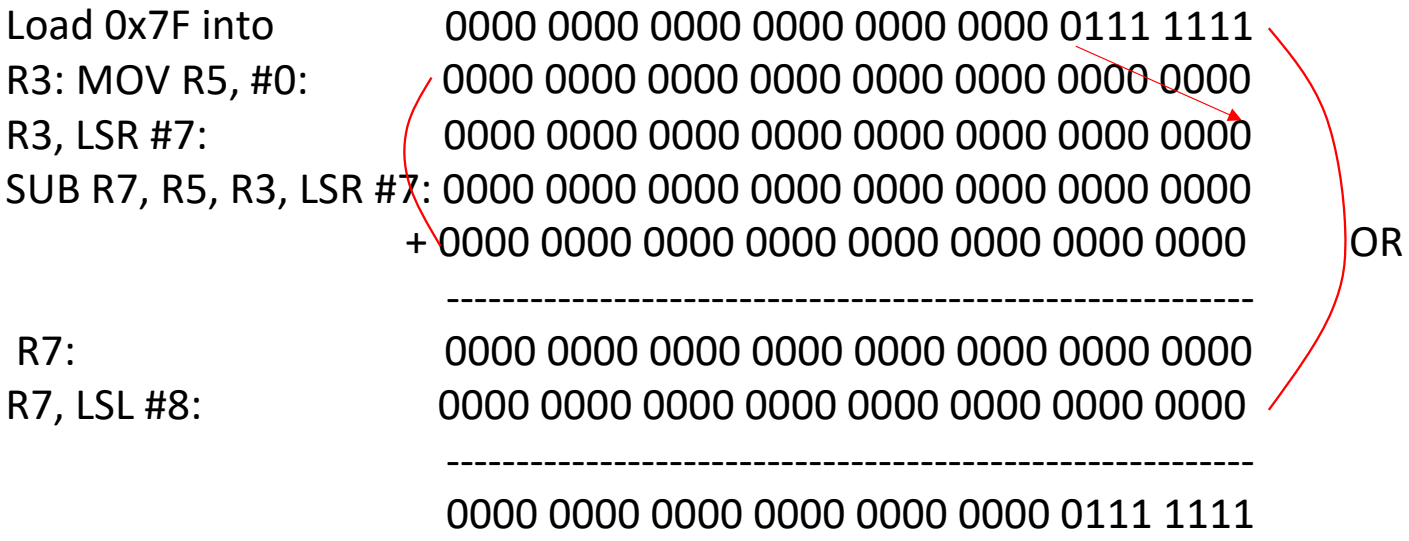
Important:

Once you are able to compare sign numbers (i.e. negative numbers too), you need to change **BHS HotFound** to **BGE HotFound** in order to allow signed comparison to take place.

```
MOV   R5,#0                ; create zero value register
SUB   R7,R5,R3,LSR #7      ; get all 1's(-ve) or 0's(+ve) by doing  0 - (R3_byte >> 7)
ORR   R3,R3,R7,LSL #8      ; OR with byte higher order 1's or 0's from bit 8 onwards
```



```
MOV   R5,#0                ; create zero value register
SUB   R7,R5,R3,LSR #7      ; get all 1's(-ve) or 0's(+ve) by doing  0 - (R3_byte >> 7)
ORR   R3,R3,R7,LSL #8      ; OR with byte higher order 1's or 0's from bit 8 onwards
```



2.4 Assembly Programming Exercise – Computing Average

Write an ARM assembly language program to compute the average value of **eight** numbers in an array.

(1) Your program should be written based on the following specifications:

a) These numbers are word-sized unsigned integers (i.e. **32-bit unsigned** numbers).

b) The eight numbers are in consecutive memory locations starting at address **`0x0100`**.

c) The average (quotient) and remainder are stored in registers **R0** and **R1** respectively.

1. *Understand how consecutive elements in an array can be accessed.*
2. *Understand how predetermine looping cycles can be implemented using a countdown counter.*
3. *Understand how unsigned and signed division by 2^n can be implemented using the logical and arithmetic shift right operation.*
4. *Understand how the use of appropriate logical operators can mask out unwanted high-order bits.*
5. *Understand the efficient use of conditional execution instructions to avoid branching.*

Write an ARM assembly language program to compute the average value of **eight** numbers in an array.

(1) Your program should be written based on the following specifications:

a) These numbers are word-sized unsigned integers (i.e. **32-bit unsigned** numbers).

b) The eight numbers are in consecutive memory locations starting at address **0x0100**.

c) The average (quotient) and remainder are stored in registers **R0** and **R1** respectively.

Note: You are free to use the partial completed VisUAL ARM assembly program template ***Tutorial 2_4-Template*** to test out your answer.

Suggested solution:

```
Start     MOV   R1,#0x100      ; initialize array pointer to start of array
          MOV   R0,#0          ; initialize cumulating register R0 to zero
          MOV   R2,#8          ; initialize loop counter for 8 loops
SumLoop   LDR   R3,[R1],#4     ; get current element and increment array pointer
          ADD   R0,R0,R3       ; add array element to current sum in R0
          SUBS  R2,R2,#1       ; decrement loop counter R2 (check Z flag)
          BNE   SumLoop        ; branch back until count is 0

GetQnR    MOV   R1,R0          ; make copy of cumulated sum in R1
          AND   R1,R1,#0x07    ; clear all bits except lowest 3 bits to get remainder in R1
          MOV   R0,R0,LSR #3   ; logical shift right by 3 bits to divide by 8 to get quotient
```

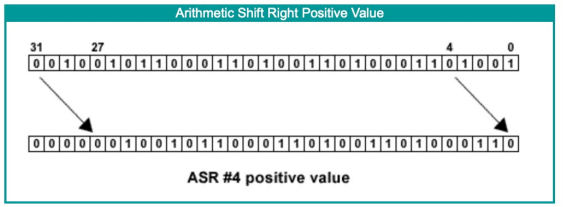
```
Start     MOV   R1,#0x100       ; initialize array pointer to start of array
          MOV   R0,#0           ; initialize cumulating register R0 to zero
          MOV   R2,#8           ; initialize loop counter for 8 loops
SumLoop   LDR   R3,[R1],#4      ; get current element and increment array pointer
          ADD   R0,R0,R3        ; add array element to current sum in R0
          SUBS  R2,R2,#1        ; decrement loop counter R2 (check Z flag)
          BNE   SumLoop         ; branch back until count is 0

GetQnR    MOV   R1,R0           ; make copy of cumulated sum in R1
          AND   R1,R1,#0x07     ; clear all bits except lowest 3 bits to get remainder in R1
          MOV   R0,R0,LSR #3    ; logical shift right by 3 bits to divide by 8 to get quotient
```

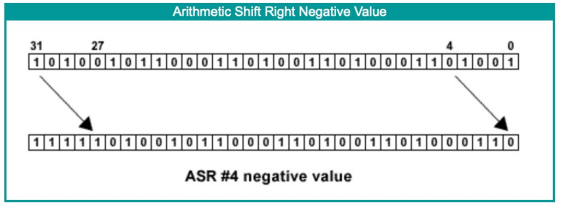
Note the following:

1. The register **R1** is used as an address pointer to the array of numbers. It is first initialized to **0x100**, the start address of the number array. The **LDR** instruction uses **R1** as an indirect register with index (and post increment) to retrieve each 32-bit number each time around the loop. 4 is added to the content in **R1** because each number (4 bytes) is stored in 4 consecutive memory locations.
2. The register **R2** is the decrementing loop counter which is initialized to 8 at the start to do 8 loops. **BNE** will continue to loop back to the **SumLoop** label until the **SUBS** instruction sets the Z flag.
3. Once the sum of all 8 numbers have been computed in register **R0**, a copy of this is made in **R1** in order to compute the remainder in **R1**. The remainder of any division by 8 (i.e. 2^3) is the least significant 3 binary bits of original number before division. This is obtained by using the **AND** operation to mask all high order bits to 0, except bits 0 to 2.
4. The positive quotient is obtained in **R0** by doing a logical right shift (**LSR**) by 3 bits.

(2) How would your program be changed if these numbers are **32-bit signed** numbers instead?

For signed numbers, Arithmetic Shift Right (**ASR**) can be used. Arithmetic right shifts are equivalent to logical right shifts (**LSR**) for positive signed numbers. Arithmetic right shifts for negative numbers (2's complement) is similar in concept to division by a power of 2^n but the negative quotients with non-zero remainder will round towards negative infinity instead of towards 0 as is usually expected (e.g. –28 / 8 gives a quotient of –4 instead of –3, because –3.5 is rounded toward –4 and not –3. As such, additional steps need to be taken to handle this rounding behaviour when using **ASR** when a negative quotient is detected.





Method #1 – Convert to positive to compute quotient and then negate quotient again

```
GetQnR  MOVS   R1,R0          ; make copy of cumulated sum in R1 (and check N flag)
        RSBMI  R0,R0,#0        ; if negative, negate to positive
        AND    R1,R1,#0x07     ; clear all bits except lowest 3 bits to get remainder in R1
        MOV    R0,R0,LSR #3    ; logical shift right by 3 bits to divide by 8
        RSBMI  R0,R0,#0        ; if quotient was earlier negative, negate to negative again
```

Note: **MOVS** is used to check if the sum is negative. If the sum is negative, then the conditional execution version of **RSBMI**, which will execute only if the N flag is set (i.e. condition is MInus). Under this situation, **RSB** will negate the sum to its positive equivalent so that the quotient can be computed as in the unsigned example earlier. However, the positive quotient must be reverted to if negative equivalent with the second **RSBMI**. Remember **RSB** unlike **SUB** implements reverse subtraction, which is (0 – R0).

(2) How would your program be changed if these numbers are **32-bit signed** numbers instead?

Method #2 – Use ASR but add #1 to negative quotient if there is a non-zero remainder

```
GetQnR  MOV     R1,R0           ; make copy of cumulated sum in R1
        MOVS    R0,R0,ASR #3    ; arithmetic shift right by 3 bits to divide by 8
        ADDMI   R0,R0,#1        ; add 1 if quotient is -ve to round quotient towards zero
        ANDS    R1,R1,#0x07     ; clear all bits except lowest 3 bits to get remainder in R1
        SUBEQ   R0,R0,#1        ; if remainder is zero, undo earlier add 1 operation quotient
```

Note: **ASR** is used to implement an arithmetic shift right, which will handle the sign of the quotient. However, if the quotient is negative, **ASR** will only give us a negative quotient that is rounded towards negative infinity. In this case, 1 is added to the negative quotient to get a rounding towards zero. This is done by the **ADDMI** instruction, which will only execute if the computed quotient is negative. However, this should only be done if there is a non-zero remainder. If there is no remainder, the addition of 1 will give us an incorrect answer. The **SUBEQ** instruction will subtract the negative quotient by 1 if the computed remainder done using the **ANDS** instruction yields a zero (i.e. Z flag set).